Introduction

Every developer has shipped code that worked perfectly on a laptop and then fell over in production. Not because the logic was wrong — the output was correct every time — but because nobody checked what happens when the input grows from a hundred rows to a hundred thousand. That gap between "it works" and "it works at scale" is one of the most common ways a quiet Tuesday afternoon turns into a six-hour incident.

This matters far beyond any one script. It's also one of the most reliable ways interviewers separate candidates who can write correct code from candidates who understand what their code actually costs. Anyone can nest two loops and get the right answer on a small example. The harder skill, and the one that shows up on real teams, is knowing *before* you ship whether that nested loop is going to be fine or going to be a pager alert.

The good news is that you don't need a computer science degree to reason about this. You need two tools that ship with Python — `timeit` and `cProfile` — and a habit of using them before performance becomes a fire.

Problem Overview

Here's the scenario, stripped down to its essence: you write a script that processes a list of records. For each record, it needs to check whether that record has already been seen — maybe to avoid duplicates, maybe to look up a related customer. You reach for a list, because a list is simple and it's what you already have. The script runs against your test data, a hundred rows or so, and it finishes instantly.

Then it runs against the real dataset. Not a hundred rows — hundreds of thousands. And the script that took a fraction of a second now takes hours, with no indication of whether it's close to done or stuck forever.

The underlying problem isn't a typo or a logic error. The code is *correct*. The problem is that the amount of work it does per item grows with the size of the data it's already processed, and nobody measured that growth curve before it mattered.

Example

Say you're deduplicating a stream of user IDs:

```
seen = []
unique_users = []

for user_id in incoming_ids:
    if user_id not in seen:      # scans the entire list
        unique_users.append(user_id)
        seen.append(user_id)
```

At 100 incoming IDs, `seen` never gets larger than 100 items, and checking `user_id not in seen` means scanning at most 100 entries. That's fast enough that you'd never notice it in testing.

At 500,000 incoming IDs, by the time you're near the end, `seen` might hold 400,000 entries, and *every single new ID* triggers a scan through all of them. You're not doing 500,000 units of work. You're doing something closer to 500,000 × 400,000 in the worst case. That's the six-hour hang.

Intuition

The way an experienced engineer catches this isn't by staring at the code and intuiting the runtime — it's by asking a specific question: **as the input doubles, does the work roughly double, or does it do something worse?**

Think of it like a party. If everyone at the party has to shake hands with everyone else, that's a lot of handshakes — and every time you add one more guest, everyone else has to shake their hand too. That's the `in` check against a list: for every new item, you re-scan everyone who came before.

Now compare that to a guest list at the door. Each person signs in once. Checking whether someone already signed in means looking at one list, not shaking every hand. That's the difference between scanning a list and checking membership in a set. Same *result* — you know who's already been seen — but a completely different amount of work as the guest list grows.

This is the intuition behind Big O notation, and it's the honest way to think about it: it's not an abstract academic exercise, it's a shorthand for "how does this get worse as data grows." O(n²) means the work grows roughly with the square of the input — double the data, quadruple the work. O(n) means the work grows in a straight line with the input — double the data, double the work. At small scale, both look instant. At production scale, only one of them survives.

The tools that turn this intuition into evidence are `timeit`, for comparing small snippets, and `cProfile`, for finding which function in a larger program is actually responsible.

Brute Force Solution

The brute-force version of the deduplication problem is the list-based approach above: keep a growing list of everything you've seen, and check membership with `in`.

Idea: maintain a list, and for every new item, scan the list to see if it's already there.

Algorithm: 1. Initialize an empty list `seen`. 2. For each incoming item, check `item in seen`. 3. If not present, add it to both `seen` and the output.

Advantages: - Trivial to write and reason about. - Preserves insertion order without extra work. - Fine for small, bounded inputs.

Disadvantages: - Membership checking against a list is O(n) per check. - Across n items, that's $O(n^2)$ total — and it's the exact shape of bug that hides in testing and explodes in production.

Time complexity: $O(n^2)$ worst case. **Space complexity:** O(n) for the `seen` list.

Optimal Solution

The fix isn't cleverer logic — it's a different data structure. Swap the list for a set.

A Python set is backed by a hash table. Checking whether an item is in a set doesn't mean scanning every existing entry; it means computing a hash and jumping directly to where that item would live. That turns an O(n) scan into an O(1) average-case lookup.

Structure	Membership check	Why
`list`	O(n)	Has to walk through items one by one
`set`	O(1) average	Hashes directly to a location

The measured difference backs this up. Using `timeit` to compare checking membership in a list of 10,000 items versus a set of the same size:

```
import timeit

data_list = list(range(10_000))
data_set = set(range(10_000))

list_time = timeit.timeit(lambda: 9_999 in data_list, number=1000)
set_time = timeit.timeit(lambda: 9_999 in data_set, number=1000)

print(f"list: {list_time:.4f}s")   # ~0.006s
print(f"set:  {set_time:.4f}s")    # ~0.001s
```

`timeit` doesn't just run the snippet once — it runs it repeatedly and averages the result, so a single lucky (or unlucky) run doesn't mislead you. The gap here — roughly 6x — only gets more dramatic as

the collection grows, because the list's cost keeps climbing while the set's stays flat.

Rewriting the deduplication logic with this in mind:

```
seen = set()
unique_users = []

for user_id in incoming_ids:
    if user_id not in seen:      # O(1) average lookup
        unique_users.append(user_id)
        seen.add(user_id)
```

Same logic, same output, one loop instead of an effectively nested one. That single change is what turns a six-hour job into a job that finishes while you're still watching it.

Python Code

Here's a complete, self-contained version, including the profiling step that would have caught the original problem:

```
import cProfile
import pstats


def dedupe_slow(ids):
    """O(n^2): list-based membership check."""
    seen = []
    unique = []
    for uid in ids:
        if uid not in seen:
            unique.append(uid)
            seen.append(uid)
    return unique


def dedupe_fast(ids):
    """O(n): set-based membership check."""
    seen = set()
    unique = []
    for uid in ids:
        if uid not in seen:
            unique.append(uid)
            seen.add(uid)
    return unique


def profile_function(func, data):
    profiler = cProfile.Profile()
```

```
    profiler.enable()
    func(data)
    profiler.disable()

    stats = pstats.Stats(profiler)
    stats.sort_stats("tottime")
    stats.print_stats(10)


if __name__ == "__main__":
    sample_data = list(range(20_000)) * 2  # duplicates included
    profile_function(dedupe_slow, sample_data)
```

Code Walkthrough

- `dedupe_slow` is the brute-force version: a plain list holding everything seen so far, checked with `in`.
- `dedupe_fast` is the optimal version: identical structure, but `seen` is a set, so the `in` check and the `add` call are both average O(1).
- `profile_function` wraps `cProfile.Profile()` around any call. `enable()` and `disable()` bracket the code you want measured — everything inside runs at normal speed while the profiler records timing per function call.
- `pstats.Stats(profiler).sort_stats("tottime")` is the critical line. Sorting by `tottime` — time spent *inside* a function, excluding time spent in functions it calls — is what tells you the real bottleneck. Sorting by `cumtime` (cumulative time, including everything a function calls) will often point you at a top-level function that's just a thin wrapper around the actual expensive work.

Dry Run

Running `profile_function(dedupe_slow, sample_data)` against 40,000 items (20,000 unique values, each duplicated once) produces output roughly shaped like this:

```
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    5.812    5.812    5.812    5.812  script.py:7(dedupe_slow)
```

Nearly all of the time is `tottime` inside `dedupe_slow` itself — because the cost is the repeated `uid not in seen` scan, not a call out to some other function. Compare that against `dedupe_fast` on the same input, and the `tottime` drops to a small fraction of a second. That side-by-side is the entire

diagnosis: same output, wildly different cost, and the cause is the data structure, not the surrounding code.

Complexity Analysis

- **Brute force:** O(n²) time, because each of the n membership checks costs O(n) in the worst case. O(n) space for the `seen` list.
- **Optimal:** O(n) time, because each membership check against a set is O(1) on average (hash collisions can degrade this, but Python's set implementation makes worst-case behavior rare in practice). O(n) space for the set — same space class as the list, just a different access pattern.

The time complexity is what actually determines whether this script finishes in seconds or hours once the input crosses from "test data" to "real data." Space complexity here is a secondary concern, though it becomes the primary one in the next section.

Alternative Solutions

If order doesn't need to be preserved, `list(set(incoming_ids))` deduplicates in a single expression, letting the set handle uniqueness and skipping the manual loop entirely. It's the same underlying algorithm, just less code, and it's a reasonable choice when you don't need to track the first-seen order.

A `dict` (using items as keys) is another valid option and was the standard trick before sets were common, since dict keys also hash. In modern Python, a set is the clearer choice when you don't need associated values.

Edge Cases

- **Empty input:** both versions return an empty list immediately; no special-casing needed.
- **All duplicates:** the brute-force version still pays the full O(n²) scanning cost even though the output is a single item, because every duplicate still triggers a full scan of `seen`.
- **All unique values:** this is actually the worst case for the list version, since `seen` grows to its maximum size and every check scans the largest possible list.
- **Unhashable items:** if you're deduping something like a list of dictionaries, the set-based fix won't work directly, since dict objects aren't hashable — you'd need to convert to a hashable representation (like a tuple of sorted items) first.
- **Very large datasets that don't fit in memory:** even the set-based fix keeps everything in memory; if the working set itself is too large, you need an external or streaming approach, not

just a different in-memory structure.

Common Mistakes

1. **Testing only on small data.** A hundred rows can't reveal an O(n²) growth curve — you need either a realistic dataset or a deliberate scaling test.
2. **Reading `cumtime` instead of `tottime` in a profiler.** A high `cumtime` often just means a function calls something expensive, not that the function itself is slow.
3. **Assuming the first function listed in profiler output is the bottleneck**, when the list is actually sorted by call order or alphabetically rather than by cost.
4. **Optimizing code that isn't the bottleneck.** Rewriting a function that takes 2% of total runtime doesn't move the needle if another function owns 90% of it.
5. **Reaching for algorithmic cleverness before checking the data structure.** Most real-world slowdowns like this one are fixed by picking the right container, not by writing smarter logic.
6. **Loading an entire large file into a list** when a generator would process it incrementally, causing memory pressure that looks like a performance bug but is actually a memory bug.
7. **Trusting a single timed run instead of using `timeit`**, which averages multiple runs and avoids being fooled by one anomalously fast or slow execution.

Interview Questions

- What's the time complexity of checking membership in a list versus a set, and why?
- How would you profile a Python script to find its actual bottleneck?
- What's the difference between `tottime` and `cumtime` in `cProfile` output?
- When would a generator be preferable to a list, and what does it cost you in exchange for the memory savings?
- Walk through why an algorithm that looks fast on a small test case can fail at scale.
- How would you deduplicate a stream of data while preserving order, and what's the complexity of your approach?

Similar Problems

- **Two Sum (LeetCode 1):** the canonical example of trading an O(n²) nested-loop check for an O(n) hash-based lookup — the exact same underlying pattern as this lesson.
- **Contains Duplicate (LeetCode 217):** directly tests whether you reach for a set instead of nested list comparisons.

- **Intersection of Two Arrays (LeetCode 349):** another membership-checking problem where set operations beat list scanning.
- **Group Anagrams (LeetCode 49):** uses hashing to avoid $O(n^2)$ pairwise comparisons, reinforcing the same data-structure-over-cleverness lesson.

Key Takeaways

- Correct code and *fast enough* code are two different bars, and only measuring tells you which one you've built.
- `timeit` compares small snippets by running them repeatedly and averaging, so you're not fooled by one lucky run.
- `cProfile` finds which function in a larger program actually deserves your attention — sort by `tottime`, not `cumtime`.
- Most real slowdowns aren't fixed with cleverer code; they're fixed by picking a data structure whose cost doesn't grow with everything that came before it.
- Generators solve the memory side of this same problem: don't hold what you don't need yet.
- Profile before it's on fire, not after.

Watch the Video

This lesson is easier to absorb watching the profiler output and the timeit comparisons run live. Catch the full walkthrough here: {LINK}

About the Series

This is part of **Fun with Learning Technology**, a series that takes real Python problems apart until they actually make sense — no unnecessary theory, just the reasoning an experienced engineer actually uses when something breaks or slows down in production.

Call To Action

If this saved you from a future six-hour production incident, give the video a like on YouTube and subscribe for more lessons like it. For deeper written breakdowns like this one, subscribe to the Substack. And if there's a Python performance topic you want covered next, drop it in the comments — I read every one.

Quick Review

Slow code trigger: what do you do before optimizing anything?

Profile first — measure with timeit/cProfile to find the actual bottleneck, don't guess.

Time complexity of a lookup in a list vs a set/dict?

List: O(n) linear scan. Set/dict: O(1) average via hashing.

Space complexity tradeoff: list comprehension vs generator?

List comprehension is O(n) memory; generator is O(1), yielding one item at a time.

Common bug when swapping list for dict/set for speed?

Forgetting duplicates get silently dropped, or losing insertion order semantics (pre-3.7 assumptions).

cProfile vs timeit — when to use which?

timeit measures a small snippet's total time; cProfile breaks a whole program into per-function call counts and time.

Interview follow-up: why can 100-row test data mislead you about performance?

Small inputs hide O(n^2)+ growth; algorithmic cost only shows up at realistic/large scale.
